

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



**Assignment : Designing and Solving Real-World
Synchronization Problems
Operating Systems : CSE 4501**

<u>Student ID: 210042112</u>

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 27, 2025

Abstract

The purpose of this assignment is to identify, analyze, and solve real-world synchronization problems using appropriate synchronization techniques.

You must create three unique synchronization problems (not including those listed in the Operating System Concepts book). These problems should reflect practical scenarios where synchronization is essential.

Contents

1	Traffic Crossroads Management	4
1.1	Problem Definition	4
1.2	Conflicting Operations/Resources	4
1.3	Potential Issues	4
1.4	Synchronization Solution	4
1.5	Deadlock Prevention	5
1.6	Deadlock Avoidance	5
1.7	Starvation Mitigation	5
2	Zombie Apocalypse Safehouse Supply Depot	5
2.1	Problem Definition	5
2.2	Conflicting Operations/Resources	5
2.3	Potential issues	6
2.4	Synchronization Solution	6
2.5	Deadlock Prevention	7
2.6	Deadlock Avoidance	7
2.7	Starvation Mitigation	7
3	Hospital Bed Allocation	7
3.1	Problem Definition	7
3.2	Conflicting Operations/Resources	8
3.3	Potential Issues	8
3.4	Synchronization Solution	8
3.5	Deadlock Prevention	8
3.6	Deadlock Avoidance	9
3.7	Starvation Mitigation	9

1 Traffic Crossroads Management

1.1 Problem Definition

A four-way traffic crossroad allows vehicles (threads) from four directions (North, South, East, West) to pass through or turn. Only one direction's vehicles can use the intersection at a time to avoid collisions, and vehicles must pass in a fair order to prevent congestion. Each direction has a queue of vehicles waiting to proceed.

1.2 Conflicting Operations/Resources

- **Shared resource:** The crossroad intersection.
- **Conflicting operations:**
 - Vehicles from different directions accessing the crossroad.
 - Managing turn signals (straight, left, right) to avoid collisions.

1.3 Potential Issues

- **Race condition:** Multiple directions attempting to access the crossroad simultaneously, risking collisions.
- **Starvation:** Vehicles from one direction (e.g., a less busy road) may wait indefinitely if other directions have heavy traffic.
- **Deadlock:** All four directions' vehicles waiting for each other, halting all movement.

1.4 Synchronization Solution

SEMAPHORES

- Use a **binary semaphore (crossroad)** to ensure exclusive access to the intersection (enforcing mutual exclusion).
- Use **counting semaphores (direction_queue[4])** for each direction to manage vehicle queues.
- Implement a **round-robin scheduler** to ensure fairness across directions.

```
1 semaphore crossroad = 1; // Binary semaphore for crossroad
2 semaphore direction_queue[4] = {0, 0, 0, 0}; // Queues for each direction
3 mutex queue_mutex;
4 int vehicle_count[4] = {0, 0, 0, 0}; // Vehicles per direction
5 int current_direction = 0; // Round-robin tracker
6
7 void vehicle(int direction, bool turn) {
8     lock(queue_mutex);
9     vehicle_count[direction]++;
10    unlock(queue_mutex);
11
12    wait(direction_queue[direction]); // Wait for turn
13    wait(crossroad); // Access crossroad
14    pass_crossroad(turn); // Move straight or turn
15    signal(crossroad); // Release crossroad
16
17    lock(queue_mutex);
18    vehicle_count[direction]--;
19    unlock(queue_mutex);
```

```

20 }
21
22 void traffic_controller() {
23     while (true) {
24         lock(queue_mutex);
25         int next_direction = (current_direction + 1) % 4;
26         if (vehicle_count[next_direction] > 0) {
27             signal(direction_queue[next_direction]); // Allow next direction
28             current_direction = next_direction;
29         }
30         unlock(queue_mutex);
31         sleep(1); // Time slice for fairness
32     }
33 }

```

1.5 Deadlock Prevention

- **Mutual exclusion:** Enforced by the crossroad semaphore, ensuring only one direction accesses the intersection.
- **Preventing circular wait:** The round-robin scheduler assigns turns to directions sequentially, eliminating circular dependencies among directions.
- **Hold and wait:** Not applicable, as vehicles release the crossroad immediately after passing.
- **No preemption:** Not applicable, as resources are not preempted.

By preventing circular wait, deadlock is avoided.

1.6 Deadlock Avoidance

The Banker's algorithm is not directly applied, as resource allocation is managed by a single semaphore and scheduler. The round-robin approach ensures safe resource allocation by checking queue status, preventing unsafe states.

1.7 Starvation Mitigation

- The round-robin scheduler cycles through directions, ensuring each gets a turn even with uneven traffic.
- The *direction_queue* semaphores prioritize queued vehicles, reducing wait times.

2 Zombie Apocalypse Safehouse Supply Depot

2.1 Problem Definition

In a zombie apocalypse, a safehouse has a supply depot with a limited number of resource kits (food, medical supplies, weapons). Survivors (threads) arrive to collect kits, with priorities based on their role: medics (highest priority, need medical kits), soldiers (medium priority, need weapons), and civilians (lowest priority, need food). The depot ensures kits are not over-allocated, and survivors are serviced fairly to maintain group survival.

2.2 Conflicting Operations/Resources

- **Shared resource:** Resource kits (medical, weapons, food).
- **Conflicting operations:**

- Survivors requesting kits based on their role.
- Restocking kits when supplies are replenished.

2.3 Potential issues

- **Race condition:** Multiple survivors requesting kits simultaneously, leading to over-allocation.
- **Starvation:** Civilians may be delayed indefinitely if medics and soldiers dominate.
- **Deadlock:** Possible if survivors and restocking processes block each other.

2.4 Synchronization Solution

SEMAPHORES and MUTEX

- Use **counting semaphores** (`medical_kits`, `weapon_kits`, `food_kits`) to track available kits (enforcing mutual exclusion for each kit type).
- Use a **binary semaphore** (`priority`) to prioritize medics, then soldiers, then civilians.
- Use a **mutex** to protect shared counters.

```

1  semaphore medical_kits = NUM_MEDICAL; // Medical kits
2  semaphore weapon_kits = NUM_WEAPONS; // Weapon kits
3  semaphore food_kits = NUM_FOOD; // Food kits
4  semaphore priority = 1; // Prioritize roles
5  mutex depot_mutex;
6  int medic_waiting = 0, soldier_waiting = 0, civilian_waiting = 0;
7
8  void survivor(string role) {
9      lock(depot_mutex);
10     if (role == "medic") {
11         medic_waiting++;
12     } else if (role == "soldier") {
13         soldier_waiting++;
14     } else {
15         civilian_waiting++;
16     }
17     unlock(depot_mutex);
18
19     if (role == "medic" || role == "soldier") {
20         wait(priority); // Medics and soldiers get priority
21     }
22
23     if (role == "medic") {
24         wait(medical_kits);
25         collect_medical_kit();
26         signal(medical_kits);
27     } else if (role == "soldier") {
28         wait(weapon_kits);
29         collect_weapon_kit();
30         signal(weapon_kits);
31     } else {
32         wait(food_kits);
33         collect_food_kit();
34         signal(food_kits);
35     }
36
37     lock(depot_mutex);

```

```

38     if (role == "medic") {
39         medic_waiting--;
40     } else if (role == "soldier") {
41         soldier_waiting--;
42     } else {
43         civilian_waiting--;
44     }
45     if (medic_waiting > 0 || soldier_waiting > 0) {
46         signal(priority); // Prioritize medics/soldiers
47     } else if (civilian_waiting > 0) {
48         signal(priority); // Allow civilians
49     }
50     unlock(depot_mutex);
51 }
52
53 void restock_supplies() {
54     lock(depot_mutex);
55     medical_kits += NEW_MEDICAL;
56     weapon_kits += NEW_WEAPONS;
57     food_kits += NEW_FOOD;
58     unlock(depot_mutex);
59 }

```

2.5 Deadlock Prevention

- **Mutual exclusion:** Enforced by the medical_kits, weapon_kits, and food_kits semaphores, ensuring exclusive access to each kit type.
- **Preventing hold and wait:** Survivors do not hold one kit while waiting for another; they wait only if their required kit is unavailable.
- **Circular wait:** Not applicable, as each survivor requests only one kit type.
- **No preemption:** Not applicable, as kits are not preempted.

By preventing hold and wait, deadlock is avoided.

2.6 Deadlock Avoidance

The Banker's algorithm is applied by treating kits as resources and survivors as processes. The priority semaphore ensures safe allocation by prioritizing medics and soldiers, while the counters (medic_waiting, soldier_waiting, civilian_waiting) ensure enough kits remain for high-priority roles, maintaining a safe state.

2.7 Starvation Mitigation

- Medics and soldiers are prioritized via **priority**, but civilians proceed when no higher-priority survivors are waiting.
- The counters ensure fair scheduling, allowing civilians access when safe.

3 Hospital Bed Allocation

3.1 Problem Definition

A hospital emergency department assigns patients (threads) to a limited number of beds. Patients arrive with varying urgency levels (critical, urgent, non-urgent). Critical patients must be assigned beds immediately, while urgent and non-urgent patients wait in prioritized queues. Beds are released when patients are discharged.

3.2 Conflicting Operations/Resources

- **Shared resource:** Available beds.
- **Conflicting operations:**
 - Assigning beds to patients based on urgency.
 - Releasing beds after patient discharge.

3.3 Potential Issues

- **Race condition:** Multiple nurses assigning beds simultaneously, leading to over-allocation.
- **Starvation:** Non-urgent patients may never get beds if critical/urgent patients keep arriving.
- **Deadlock:** Possible if bed assignment and release processes wait for each other.

3.4 Synchronization Solution

MONITORS

- Use a **monitor** to manage bed allocation and release (enforcing mutual exclusion).
- Use **condition variables** to handle waiting patients by urgency level.

```
1  monitor hospital_beds {
2      int available_beds = MAX_BEDS;
3      condition critical_wait, urgent_wait, nonurgent_wait;
4
5      procedure assign_bed(int urgency) {
6          if (available_beds == 0) {
7              if (urgency == CRITICAL) {
8                  wait(critical_wait);
9              } else if (urgency == URGENT) {
10                 wait(urgent_wait);
11             } else {
12                 wait(nonurgent_wait);
13             }
14         }
15         available_beds--;
16         use_bed();
17     }
18
19     procedure release_bed() {
20         available_beds++;
21         if (!signal(critical_wait)) { // Prioritize critical
22             if (!signal(urgent_wait)) { // Then urgent
23                 signal(nonurgent_wait); // Finally non-urgent
24             }
25         }
26     }
27 }
```

3.5 Deadlock Prevention

- **Mutual exclusion:** Forced by the monitor, ensuring that only one thread modifies *available_beds* at a time.

- **Preventing hold and wait:** Patients do not hold one bed while waiting for another; they wait only if no beds are available and release occurs independently.
- **Circular wait:** Not applicable, as there is no cycle of resource dependencies.
- **No preemption:** Not applicable, as beds are not preempted from patients.

By **preventing hold and wait**, deadlock is avoided.

3.6 Deadlock Avoidance

The Banker's algorithm can be applied by treating beds as resources and patients as processes. Before assigning a bed, the system checks if the allocation keeps the system in a safe state (i.e., enough beds remain for critical patients). The monitor implicitly ensures this by prioritizing critical patients and queuing others, mimicking a safe allocation strategy.

3.7 Starvation Mitigation

- Critical patients are prioritized, followed by urgent, then non-urgent, ensuring fairness.
- The release process signals waiting patients in priority order, reducing delays for non-urgent cases.

The End.